

Pascal-S 编译器 - 测试计划

汇报人：金培功

1. 测试总体设计思路

本项目的测试目标是“验证 **Pascal-S** 到 **C** 编译器前端与翻译器的工作正确性与健壮性”。为了证明系统能够正常工作，测试体系不仅要验证“正确的代码能否被正确编译和执行”，还要验证“错误的代码能否在正确的阶段被拦截并给出精准的报错信息”。

我们将采用**分阶段测试与端到端（E2E）测试相结合**的策略，具体分为以下五个验证层次：

1. **前端模块测试**：词法切分正确性、语法归约与 AST 树结构校验。
2. **语义模块测试**：符号表作用域检查、类型推导与语义规则拦截校验。
3. **代码生成测试**：AST 到 C 语言结构的映射正确性验证。
4. **错误熔断测试**：词法或语法错误数量超出阈值时，编译器能及时终止处理，避免输出大量无意义的后续错误。
5. **端到端闭环测试**：将生成的 C 代码使用标准 C 编译器（如 GCC）编译执行，通过对比输入输出（I/O）验证最终语义等价性。

2. 各阶段测试点设计

根据编译器的处理管线，我们设计了以下五个阶段的测试内容：

2.1 词法分析阶段

重点验证源程序能否正确转化为 Token 流，及对非法字符的容错能力。

- **正常情况**：正确识别保留字（不区分大小写）、标识符、整数/实数/字符常量、各类运算符及界符。正确跳过空白符和 {...} 注释。
- **边界情况**：连续多个空白符或嵌套换行的解析；大小写混合的标识符（如 pRoGrAm 统一转为 program）。
- **异常情况**：
 - 出现文法外的非法字符（如 @, #）。
 - 未闭合的注释（如只有 { 没有 }）。
 - 未闭合的字符常量（如 'a 缺少右侧引号）。

2.2 语法分析与 AST 构造阶段

重点验证语句结构的完整性以及 AST 节点生成的准确性。

- **正常情况**：支持完整的程序结构（program...begin...end.），各种声明（const, var, array），以及基本语句（if-then-else, for-to-do, while-do, break, read/write，赋值，函数/过程调用）。
- **边界情况**：极深的嵌套结构（多层 if 或多层 for 循环）；复杂的混合表达式（如关系运算、乘除法和一元负号的组合）；空语句或仅包含一条语句的复合块。
- **异常情况**：
 - 缺失必要符号（如漏写分号；、缺失匹配的括号）、缺失 end 等关键字）。
 - 不符合文法的 Token 序列（如连续两个操作符 $a + * b$ ）。

2.3 语义分析阶段

重点验证符号表作用域管理和类型检查规则。这是错误排查的重灾区。

- **正常情况**：同名标识符在不同作用域的正确屏蔽；值参和引用参数（var）的正确传递；符合规范的数组下标访问。
- **边界情况**：全局变量与局部变量同名；函数体内对函数名自身的合法赋值（作为返回值）。
- **异常情况**：
 - **声明错误**：变量/函数未声明先使用；同一作用域内重复声明。
 - **类型错误**：向常量（const）重新赋值；赋值语句左右类型不兼容（如 `integer := boolean`）；if 条件不是布尔类型。
 - **调用错误**：数组未按数组方式使用（直接当普通变量用）；函数/过程调用时参数个数或类型不匹配。

2.4 错误熔断阶段

重点验证当源程序错误数量超出预设阈值时，编译器能及时终止处理。

- **正常情况**：错误数量低于阈值，本阶段能完整运行至结束并报告全部错误。
- **边界情况**：错误数量恰好达到阈值时的临界行为。
- **异常情况**：词法或语法错误数量超出阈值，编译器输出“错误过多，停止扫描”提示并中止当前阶段的后续处理。

2.5 目标代码生成与端到端阶段

重点验证输出的 C 代码是否与原 Pascal-S 语义绝对等价。

- **正常情况**：数据类型的等价映射（boolean 映射为 stdbool.h 或 int，数组映射为 C 数组）；Pascal-S 的子程序正确映射为 C 的函数（void 或带返回值）；read/write 格式串正确生成。
- **边界情况**：Pascal-S 数组下标可能非零起始（如 array[5..10]），需验证生成的 C 代码下标偏移计算是否越界或错位。

3. 典型测试用例示例

为了更直观地展示验证标准，以下提取了部分典型测试用例：

测试阶段	测试场景	测试输入 (Pascal-S)	预期输出 / 判定标准
词法异常	未闭合注释	program test; { this is a comment begin end.	报错：【词法错误】行 1, 列 15：未闭合的注释。继续或停止扫描。
语法正常	大小写混用	VaR X : InTeGeR;	成功归约，AST 对应变量的声明节点，标识符存储为 x，类型为 integer。
语义异常	重复声明	var a: integer; a: real;	报错：【语义错误】行 X：标识符 ‘a’ 在当前作用域已重复声明。
语义异常	常量赋值	const PI = 3.14; begin PI := 3.15; end.	报错：【语义错误】行 X：不能向常量 ‘PI’ 赋值。
语义异常	参数不匹配	procedure p(a: integer); begin end; begin p(1, 2); end.	报错：【语义错误】行 X：过程 ‘p’ 调用参数个数不匹配，期望 1 个，实际 2 个。
端到端测试	循环与累加	var i, sum: integer; begin sum := 0; for i := 1 to 5 do sum := sum + i; write(sum); end.	成功生成等价的 C 代码。使用 GCC 编译运行 C 程序后，控制台输出 15。